Project Report

Minimum Troop Placement in a Tree

Design and Analysis of Algorithms

Student Name: Arnav Tiwary, Swarnav Kalita

Roll Number: bmat2410, bmat2435

Institution: Indian Statistical Institute

Course: Design and Analysis of Algorithms

October 2026

Contents

1 Abstract **3**

2 Introduction **3**

3 Problem Statement **4**

3.1 Formal description 4

3.2 Graph representation 4

4 Preliminary Graph-Theoretic Results **5**

4.1 The given graph is a tree 5

4.2 Rooting the tree and defining depth 5

4.3 DFS visits every vertex without deleting vertices 6

4.4 A tree with n vertices has $n-1$ edges 7

5 Algorithm Design **8**

5.1 Overview 8

5.2 Adjacency List 9

5.3 DFS Traversal 9

5.4 Dynamic Programming States Definition 9

5.5 Derivation of Dynamic Programming Transitions 10

5.5.1 When there is 1 or 2 troops at the node 11

5.5.2 When there are 0 troops and parent node has 2 troops 11

5.5.3 Case 3: Zero troops and a parent without two troops 12

5.6 Initialisation and Final Answer 13

6 Complete C++ Implementation **13**

7 Correctness Proof **21**

7.1 Children are handled first 21

7.2 All possible assignments are covered 21

7.3 All DP states are optimal 22

7.4 Global Optimality 22
7.5 Correctness of Reconstruction 23

8 Complexity Analysis **23**
8.1 Reading Input And Building List Of Adjacency Lists 23
8.2 DFS traversal 24
8.3 Reverse order 25
8.4 DP Table Initialization 25
8.5 Complexity of the DP transitions 25
8.5.1 Case 1: Vertex has 1 or 2 troops 26
8.5.2 Case 2: No troops. Parent has 2 troops 26
8.5.3 Case 3: 0 troops and parent does not have 2 troops 26
8.6 Reconstruction 28
8.7 Overall time complexity 28
8.8 Overall space complexity 29

9 Conclusion **29**

1 Abstract

This project considers the problem of how many military personnel to place in towns connected by roads in a tree structure. Each town must hold 0, 1 or 2 people. Any town which holds 0 people should have exactly one neighbour with 2 people.

We can represent this road network as an undirected tree. We will use a dynamic programming approach on a rooted tree. We first create an adjacency list representation of the graph, then perform a depth first search to find the parent-child relationships. Finally we process the vertices in reverse order of the DFS and compute an optimal solution for subtrees based on different combinations of values for current vertex and parent.

This report provides the graph theory properties needed for the algorithm, the dynamic programming transitions, a proof by induction on the height of subtrees and a full C++ implementation. The implementation also includes a detailed analysis of its time and space complexity.

The worst case time complexity of this implementation is $\Theta(n^2)$ and the space complexity is $O(n)$, where n is the number of towns.

2 Introduction

Graph Algorithms allow us to systematically solve optimisation problems on networks. If the graph is a tree, the absence of cycles means we can break the problem down into sub-problems. In this problem we must place troops within a network of towns. Each town can have 0, 1 or 2 troops. Any towns which have 0 troops must have exactly one neighbour with 2 troops. We must place the minimum number of troops. A brute force approach would involve looking at all possible ways to assign the number of troops to each town. As there are 3 possible troop numbers for each of the n towns we have 3^n assignments. This would take a long time. We utilise the fact that the graph is a tree. Using dynamic programming we can fix the number of troops in a vertex and its parent. This allows us to relate the optimal solution for a vertex and its subtree to optimal solutions of the children. Our aims are:

1. Determine what graph theoretic properties the provided road network has.
2. Design an algorithm to minimise the number of troops using dynamic programming.
3. Provide a rigorous proof of our algorithm.
4. Analyse the time and space complexity of our implementation.
5. Provide full C++ code

3 Problem Statement

3.1 Formal description

There are n towns in country XYZ. The towns are connected with $n-1$ roads. There is exactly one path between any pair of towns.

Two towns which are directly connected with a road are called neighbors.

We have to assign each town u a number of troops

$$x_u \in \{0, 1, 2\}.$$

The assignment must satisfy the following condition:

$$\boxed{x_u = 0 \implies |\{v \in N(u) : x_v = 2\}| = 1,}$$

where $N(u)$ denotes the set of neighbors of u (i.e. those towns adjacent to it).

This means if the town has zero troops, exactly one of the neighboring towns will have two troops.

If the town has either one or two troops, there are no further constraints on how many of its neighbors have two troops.

Our aim is to minimize

$$\boxed{\sum_{u \in V} x_u,}$$

subject to the constraint above.

3.2 Graph representation

We represent the country as an undirected graph

$$G = (V, E),$$

where V is the set of towns and E is the set of roads.

The input satisfies

$$|V| = n, \qquad |E| = n - 1.$$

The problem also guarantees that every pair of vertices is connected by a unique path. We establish below that this property makes G a tree.

4 Preliminary Graph-Theoretic Results

4.1 The given graph is a tree

Definition 1. An undirected graph is a tree if it is connected and contains no cycles.

We show that the given graph satisfies both conditions.

Connectedness. All towns can be accessed from any other town using the roads. For any pair of towns $u, v \in V$ there exists a path between u and v. Therefore, G is connected.

Absence of cycles. We will assume the opposite, and say that there is a cycle in G. Pick two different vertices in the cycle, these will be called u and v. We can then move from u to v either way around the cycle:

This means there are two different paths from u to v, which contradicts the fact that every pair of towns have a unique path connecting them.

Thus there are no cycles in G

As G is connected and has no cycles it is therefore a tree. Hence,

$$\boxed{G \text{ is a tree.}}$$

4.2 Rooting the tree and defining depth

We select vertex 1 as the root.

The depth of a vertex u, denoted by $d(u)$, is the number of edges in the unique path from the root to u. In particular we have

$$d(1) = 0.$$

Each non-root vertex u has a unique parent $p(u)$ which is the vertex immediately before u on the unique path from the root to u.

The children of u are those vertices which have parent u.

The subtree with root u, denoted T_u, consists of u together with all of its descendents.

A tree has the property that all non-root vertices have a unique parent. For any edge either one vertex is the parent of the other. Having a second edge between two vertices which have a connecting path would create a cycle which cannot happen in a tree. This means we can decompose our optimisation problem into subproblems.

4.3 DFS visits every vertex without deleting vertices

We prove that each vertex is visited once.

All vertices are visited. Initially the root (1) is pushed onto the stack and will be visited.

When we visit u, we look at all of its neighbors. We ignore the parent and push all its children onto the stack. As we continue until the stack is empty, all vertices that have been pushed on the stack will have been visited.

Assume there exists a vertex v that has not been visited. As the tree is a connected graph there exists a unique path between the root and v,

$$1 = v_0, v_1, ..., v_k = v.$$

The root $v_0 = 1$ has been visited. When visiting v_i, we push its child v_{i+1} onto the stack. This node will be visited at some point. Using induction along the path we see that $v_k = v$ must be visited. This contradicts our assumption. Therefore all vertices are visited.

Every vertex is visited exactly once. The root has no parent and all other vertices have one parent. When processing vertex u the parent is skipped and the children are pushed onto the stack.

If any vertex v has been discovered more than once this means there are two distinct paths from the root to vertex v. The union of these paths must contain a cycle which contradicts the graph being a tree.

Therefore, every vertex is discovered and processed exactly once.

No edge is missed. Consider any edge (u, v) of the tree. When the algorithm processes u, it examines the edge connecting u and v.

There are two possibilities:

- **Case 1:** v is the parent of u. The algorithm skips this edge because it has already been traversed when u was discovered from v.
- **Case 2:** v is a child of u. The algorithm pushes v onto the stack, thereby discovering v.

In general, any edge that is not used would create a cycle between the two vertices, since the other end of the edge has already been found via a different route. However, since this is a tree this is impossible. We can therefore conclude that every edge will be used either to find a new vertex or return to a parent vertex.

Conclusion. The algorithm visits each vertex once and accounts for all edges. This means that it will correctly perform a DFS traversal of the tree without altering the graph.

4.4 A tree with n vertices has $n-1$ edges

We prove this property by induction on the maximum depth of a rooted tree.

Let

$$L_k = \{u \in V : d(u) = k\}$$

denote the set of vertices at depth k.

Suppose the maximum depth is h. The depth levels partition the vertex set:

$$V = L_0 \cup L_1 \cup \cdots \cup L_h,$$

where the sets are pairwise disjoint.

Since the root is the only vertex at depth zero,

$$|L_0| = 1.$$

Base case: $h = 0$.

If the maximum depth is zero, the tree consists only of the root. Therefore,

$$|V| = 1, \qquad |E| = 0.$$

Hence,

$$|E| = |V| - 1.$$

Inductive hypothesis.

Assume that every rooted tree of maximum depth at most $h-1$ has one fewer edge than vertices.

Inductive step.

Consider a rooted tree, with depth h. Remove all vertices at depth h and their connected edges.

All vertices at depth h must be leaves, since if they had children, those children would be at depth $h+1$ which would mean that the tree is not of depth h. We know we have a tree and also that it is of depth at most $h-1$. There are n' total vertices so (by induction hypothesis) there are

$$n' - 1 \tag{1}$$

edges. Let $m = |L_h|$ be the number of vertices at depth h. Every leaf will only have one edge connecting it to its parent. Therefore the original tree has a total of

$$n = n' + m \tag{2}$$

vertices and

$$|E| = (n' - 1) + m \tag{3}$$

edges. That means

$$|E| = n' + m - 1 = n - 1 \tag{4}$$

Hence our result has been shown for depth h. The result now follows by induction on the maximum depth Therefore the number of edges in any finite tree has been proved.

$$\boxed{|E| = n - 1}$$

5 Algorithm Design

5.1 Overview

We will use dynamic programming on the rooted tree.

The algorithm can be split into three parts:

1. **Constructing the tree and doing DFS:**

   We use an adjacency list to store the edges. DFS is performed from vertex 1 which gives us the parent and depth of each vertex as well as the order we visit them.

2. **Dynamic programming:**

   We reverse the order and perform dynamic programming on all the nodes. For each node we calculate the minimum cost for all combinations of troop counts from the parent and current node.

3. **Getting the answer for the root node:**

   As there is no parent for the root node we use a sentinel value for the parent. We then take the minimum of the three possible troop counts.

The main issue is dealing with nodes with no troops. This node must have only one neighbour that has 2 troops and the transition depends on whether the parent already has 2 troops.

5.2 Adjacency List

The tree is stored using

```
vector<vector<int>> adj(n + 1)
```

where `adj[u]` contains all the neighbors of vertex u.

For each undirected edge (u, v), we add v to `adj[u]` and u to `adj[v]`.

As there are $n - 1$ edges, there will be

$$2(n - 1)$$

elements in the adjacency list. Hence the graph will take up $O(n)$ space.

5.3 DFS Traversal

Three arrays are used:

- `parent[u]`: the parent of vertex u.
- `depth[u]`: the depth of vertex u.
- `order`: the order of vertices visited during the DFS.

The root vertex has

$$\texttt{parent[1]} = 0, \qquad \texttt{depth[1]} = 0.$$

Each child vertex v of u has

$$\texttt{parent[v]} = u, \qquad \texttt{depth[v]} = \texttt{depth[u]} + 1.$$

The vertices are recorded in the order they are visited in the traversal. This means that for each parent vertex, it will appear in the `order` array before any of its children.

This order is reversed before the dynamic programming transitions. This ensures that for each parent vertex, each of its children will appear first. This means that when processing a vertex, all of the children states will have been calculated.

Here is the formatted LaTeX, keeping the wording the same and correcting only the formatting and mathematical notation.

5.4 Dynamic Programming States Definition

Let T_u be the subtree rooted at vertex u.

Let us define the following DP states:

$$\boxed{dp[u][p][x]}$$

the minimal sum of the number of soldiers in subtree T_u, where:

- Vertex u contains exactly x soldiers.
- Vertex-parent of u contains exactly p soldiers.
- All vertices in the subtree T_u meet the soldier-placing condition.

The number of soldiers in the current vertex is:

$$x \in \{0, 1, 2\}.$$

The number of soldiers in the parent vertex (if vertex is not a root):

$$p \in \{0, 1, 2\}.$$

But in case of the root we add a special value $p = 3$ that indicates that there is no parent. We see that the number of soldiers cannot be equal to 3.

$$p \in \{0, 1, 2, 3\}.$$

In total we have 4 values for parent states and 3 for current vertex. So each vertex has 12 DP states.

If the state is unreachable, we will assign it a special value:

$$\texttt{INF} = \frac{\texttt{LLONG_MAX}}{4}.$$

The value stored in the state represents the sum of all costs including the soldiers placed in u, but excluding the soldiers in the parent vertex and all vertices outside of the subtree T_{u_0}.

5.5 Derivation of Dynamic Programming Transitions

Let $C(u)$ denote the set of children of vertex u. Consider a vertex u, its parent state p, and its troop count x. Let us consider three cases:

5.5.1 When there is 1 or 2 troops at the node

Let us suppose

$$x \in \{1, 2\}.$$

The constraint applies only to vertices containing zero troops. Therefore, the condition at u is automatically satisfied.

Now considering a child node v of u. The parent of node v is u, and this node has x troops. So the parent should be in state x.

Our node can have 0, 1 or 2 troops. Taking the minimum

$$\min\{dp[v][x][0], dp[v][x][1], dp[v][x][2]\}.$$

The subtrees for children are separate from each other. The only connection they have to the rest of the tree is through u. Once we know how many troops are at node u we can optimally solve the subproblem for each child.

We can take the total cost from each child and the x troops at node u_0 to get

$$\boxed{dp[u][p][x] = x + \sum_{v \in C(u)} \min_{y \in \{0,1,2\}} dp[v][x][y]}$$

for v_1.

The parent troop count p does not affect this transition because u already has at least one troop.

5.5.2 When there are 0 troops and parent node has 2 troops

Let us suppose

$$x = 0, \qquad p = 2.$$

The parent of node u already has 2 troops. Therefore one neighbour of node u already has 2 troops.

We know that node u must have one and only one neighbour with 2 troops. So none of its children can have two troops.

This means the children of node u must all be 0 or 1. Taking the minimum values

$$\min\{dp[v][0][0], dp[v][0][1]\}.$$

As the current node has 0 troops there is no additional cost at node u_0.

We can now say

$$\boxed{dp[u][2][0] = \sum_{v \in C(u)} \min\{dp[v][0][0], dp[v][0][1]\}.}$$

This will only apply if the child state chosen is valid. If the child state cannot be chosen then it will have an infinite cost.

The parent of u is not included in the sum because it lies outside T_u.

5.5.3 Case 3: Zero troops and a parent without two troops

Suppose

$$x = 0, \qquad p \neq 2.$$

This includes when the root is considered. The parent will be in state $p = 3$.

Therefore the parent does not have 2 troops. Exactly one child of u will have 2 troops.

We can iterate through each child w and find the one that is assigned two troops. This will contribute

$$dp[w][0][2],$$

because the parent has no troops, i.e. u.

All other children $v \neq w$ cannot have 2 troops, otherwise there will be more than one child of u with 2 troops.

The minimum contribution is

$$\sum_{\substack{v \in C(u) \\ v \neq w}} \min\{dp[v][0][0], dp[v][0][1]\}.$$

So the total cost of choosing w is

$$C(w) = dp[w][0][2] + \sum_{\substack{v \in C(u) \\ v \neq w}} \min\{dp[v][0][0], dp[v][0][1]\}.$$

Then we take the min over all choices,

$$\boxed{dp[u][p][0] = \min_{w \in C(u)} C(w)}$$

for $p \neq 2$.

If u has no children then this is impossible. Neither the parent nor any children can give the required neighbour of 2 troops. So this will be worth `INF`.

5.6 Initialisation and Final Answer

Initially all dp states will be set to `INF`.

The vertices will be processed in reverse DFS order so children will have been processed first.

Once all vertices have been processed the root will have state $p = 3$ as its parent. The root can have 0, 1 or 2 troops, so the final answer is

$$\boxed{\text{OPT} = \min\{dp[1][3][0], dp[1][3][1], dp[1][3][2]\}}$$

If all 3 options are `INF` then there is no valid assignment, however this is practically impossible as we can always assign one troop to every vertex. Otherwise the minimum of the three options is the minimum total number of troops required.

6 Complete C++ Implementation

The following implementation uses an adjacency list, iterative DFS, and dynamic programming on the rooted tree.

```
#include <bits/stdc++.h>
using namespace std;

const long long INF = LLONG_MAX / 4;

vector<vector<int>> input(int n)
{
    vector<vector<int>> adj(n + 1);

    for(int i = 0; i < n - 1; i++)
    {
        int u, v;
        cout << "Enter edge " << i + 1 << ": ";
        cin >> u >> v;

        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    return adj;
}

void dfs(vector<vector<int>>& adj,
```

```
          vector<int>& parent,
          vector<int>& depth,
          vector<int>& order)
{
    stack<int> st;
    st.push(1);

    parent[1] = 0;
    depth[1] = 0;

    while(!st.empty())
    {
        int u = st.top();
        st.pop();

        order.push_back(u);

        for(int v : adj[u])
        {
            if(v == parent[u])
                continue;

            parent[v] = u;
            depth[v] = depth[u] + 1;
            st.push(v);
        }
    }
}

void dynamicProgramming(
    vector<vector<int>>& adj,
    vector<int>& parent,
    vector<int>& depth,
    vector<int>& order,
    vector<vector<array<long long, 3>>>& dp)
{
    reverse(order.begin(), order.end());

    for(int u : order)
    {
        // p = 0, 1, 2: parent's troop count
        // p = 3: u is the root
        for(int p = 0; p <= 3; p++)
        {
            // x = troop count at u
```

```
        for(int x = 0; x <= 2; x++)
        {
            long long best = INF;

            if(x == 0)
            {
                // u has zero troops.
                // Exactly one neighbor must have two troops.

                if(p == 2)
                {
                    // Parent already has two troops.
                    // No child can have two troops.

                    best = 0;

                    for(int v : adj[u])
                    {
                        if(v == parent[u])
                            continue;

                        best += min(dp[v][0][0],
                                    dp[v][0][1]);
                    }
                }
                else
                {
                    // Parent does not have two troops.
                    // Exactly one child must have two troops.

                    for(int chosen : adj[u])
                    {
                        if(chosen == parent[u])
                            continue;

                        // Choose this child to have two troops.
                        long long cost = dp[chosen][0][2];

                        // Every other child must have 0 or 1 troops.
                        for(int v : adj[u])
                        {
                            if(v == parent[u] || v == chosen)
                                continue;

                            cost += min(dp[v][0][0],
```

```
                                        dp[v][0][1]);
                        }

                        best = min(best, cost);
                    }
                }
            }
            else
            {
                // u has one or two troops.
                // Its own condition is automatically satisfied.
                // Each child can have 0, 1, or 2 troops.

                best = 0;

                for(int v : adj[u])
                {
                    if(v == parent[u])
                        continue;

                    best += min({
                        dp[v][x][0],
                        dp[v][x][1],
                        dp[v][x][2]
                    });
                }
            }

            // Add the troops stationed at u.
            if(best < INF)
                dp[u][p][x] = x + best;
            else
                dp[u][p][x] = INF;
        }
    }
}

// Reconstruct an optimal troop allocation using the DP table.
void reconstruct(
    int u,
    int p,
    int x,
    vector<vector<int>>& adj,
    vector<int>& parent,
```

```
    vector<vector<array<long long, 3>>>& dp,
    vector<int>& troops)
{
    // Store the number of troops assigned to u.
    troops[u] = x;

    // Case 1: u has one or two troops.
    if(x > 0)
    {
        for(int v : adj[u])
        {
            if(v == parent[u])
                continue;

            // Choose the optimal troop count for child v.
            int chosenX = 0;

            for(int y = 1; y <= 2; y++)
            {
                if(dp[v][x][y] < dp[v][x][chosenX])
                    chosenX = y;
            }

            reconstruct(v, x, chosenX,
                        adj, parent, dp, troops);
        }
    }

    // Case 2: u has zero troops and its parent has two troops.
    else if(p == 2)
    {
        for(int v : adj[u])
        {
            if(v == parent[u])
                continue;

            // Children can have only zero or one troop.
            int chosenX = 0;

            if(dp[v][0][1] < dp[v][0][0])
                chosenX = 1;

            reconstruct(v, 0, chosenX,
                        adj, parent, dp, troops);
        }
```

```
    }

    // Case 3: u has zero troops and its parent does not
    // have two troops. Exactly one child must have two troops.
    else
    {
        int chosenChild = -1;
        long long bestCost = INF;

        // Find the child that should receive two troops.
        for(int chosen : adj[u])
        {
            if(chosen == parent[u])
                continue;

            long long cost = dp[chosen][0][2];

            for(int v : adj[u])
            {
                if(v == parent[u] || v == chosen)
                    continue;

                cost += min(dp[v][0][0],
                            dp[v][0][1]);
            }

            if(cost < bestCost)
            {
                bestCost = cost;
                chosenChild = chosen;
            }
        }

        // Reconstruct the allocation for each child.
        for(int v : adj[u])
        {
            if(v == parent[u])
                continue;

            int chosenX;

            if(v == chosenChild)
            {
                chosenX = 2;
            }
```

```
            else
            {
                chosenX = 0;

                if(dp[v][0][1] < dp[v][0][0])
                    chosenX = 1;
            }

            reconstruct(v, 0, chosenX,
                        adj, parent, dp, troops);
        }
    }
}

int main()
{
    cout << "Enter the number of vertices in your tree: ";

    int n;
    cin >> n;

    if(n <= 0)
    {
        cout << "The number of vertices must be positive.\n";
        return 0;
    }

    cout << "Enter the " << n - 1 << " edges (u v):\n";

    vector<vector<int>> adj = input(n);

    vector<int> parent(n + 1, 0);
    vector<int> depth(n + 1, 0);
    vector<int> order;

    dfs(adj, parent, depth, order);

    vector<vector<array<long long, 3>>> dp(
        n + 1, vector<array<long long, 3>>(4)
    );

    for(int u = 0; u <= n; u++)
    {
        for(int p = 0; p <= 3; p++)
            dp[u][p].fill(INF);
```

```
    }

    dynamicProgramming(adj, parent, depth, order, dp);

    // Root 1 has no parent, represented by p = 3.
    long long ans = min({
        dp[1][3][0],
        dp[1][3][1],
        dp[1][3][2]
    });

    cout << "\nMinimum number of troops: ";

    if(ans >= INF)
    {
        cout << "No valid assignment exists\n";
        return 0;
    }

    cout << ans << '\n';

    // Reconstruct an optimal allocation.
    int rootX = 0;

    for(int x = 1; x <= 2; x++)
    {
        if(dp[1][3][x] < dp[1][3][rootX])
            rootX = x;
    }

    vector<int> troops(n + 1, 0);

    reconstruct(1, 3, rootX,
                adj, parent, dp, troops);

    cout << "\nTroop allocation:\n";

    for(int u = 1; u <= n; u++)
    {
        cout << "Vertex " << u << ": "
             << troops[u]
             << (troops[u] == 1 ? " troop" : " troops")
             << '\n';
    }

```

```
339     return 0;
340 }
```

7 Correctness Proof

We can prove that the algorithm gives the minimal number of soldiers needed to guard the tree, given that the input is a valid tree with vertices numbered 1 through n.

7.1 Children are handled first

Lemma 1. *All children are processed before their parents.*

Proof. When performing a DFS, all children are visited after their parents are added to the list. This means that all parents will appear before their children in the original list.

As we reverse the list before calculating the DP table, all children appear before their parents. This means that when processing vertex u, all children of this vertex have already had their DP states calculated. □

7.2 All possible assignments are covered

Lemma 2. *For all states in the DP, all possible valid assignments are considered.*

Proof. We consider a vertex u with parent state p and number of troops x.

If $x \in 1, 2$, then the condition at u is automatically satisfied. All possible values of the number of troops in each of the children of the vertex are considered, with 3 choices and the minimum cost being chosen, hence all possible assignments are considered.

If $x = 0$ and $p = 2$, then we know that the parent vertex has already allocated 2 troops to one of its neighbours, so none of the children can have 2 troops. This means that for each child of the vertex, only 0 or 1 troops can be allocated, therefore all valid assignments are considered.

If $x = 0$ and $p \neq 2$, then exactly one child must have 2 troops allocated. The recurrence covers all children by considering each one as the unique choice of the node with 2 soldiers , and all other children have 0 or 1 troops. All valid assignments will fall into one of these cases.

In the last case it may be the case that there are no children. In this case the state is invalid as no troops can be allocated to the required neighbour (no troops can be allocated by the parent or children).

Therefore, all valid assignments are considered. □

7.3 All DP states are optimal

Lemma 3. *The algorithm calculates the correct minimum number of soldiers needed to be assigned to vertex u with parent state p and an amount of soldiers x in T_u such that the conditions of $dp[u][p][x]$ are met. If there is no assignment, the state will be marked as INF.*

Proof. We prove this by induction on the height of the subtree rooted at u.

Base case:

Assume that u is a leaf node. This means that its subtree consists of just u. If either $x = 1$ or $x = 2$ then the condition $p = 0$ will always be satisfied, therefore the minimum cost is 1. If instead $x = 0$, then the parent is the only possible neighbour. If $p = 3$ then the state is possible and costs 0. If $p = 4$ then there are no neighbours that have two soldiers. Therefore, the state is impossible. This covers all cases that the algorithm considers so the states are calculated correctly for leaves.

Inductive step:

Assume that all DP states have been correctly calculated for subtrees with height less than or equal to $h - 1$. Take any vertex u with subtree height h. This means all of the children of u will have subtree heights less than or equal to $h - 1$ and therefore the induction hypothesis holds and all DP states within the child subtrees are correctly calculated. From the first lemma we know that u will only be processed once all of its children have been processed. From the second lemma we know that the transitions will check every possible assignment and pick the one with the lowest cost. Therefore all DP states for u will be correctly calculated. Thus using induction on the subtree heights we can see that all DP states in the tree are either optimal or marked as impossible. □

7.4 Global Optimality

Theorem 1. *The algorithm will always return the minimal number of troops for any valid assignment.*

Proof. We know that the root node will have one of the three possible troop counts for any valid assignment:

$$x \in \{0, 1, 2\}.$$

Because the root node has no parents, the parent state is represented by $p = 3$.

By the previous lemma, we know that the values

$$dp[1][3][0],\ dp[1][3][1],\ dp[1][3][2]$$

will give us the minimal costs for each assignment, or `INF` if there isn't a valid assignment. Therefore, since any valid assignment will fall into one of these three cases, we can take the minimum of these three values to get the minimal cost:

$$\boxed{OPT = \min(dp[1][3][0], dp[1][3][1], dp[1][3][2]).}$$

If all states are invalid then there is no valid assignment. This means that the algorithm will always return the global optimum if there is a valid assignment, and otherwise correctly states there is no valid assignment. (In practice however, there will always be a valid assignment as placing 1 soldier in each node is always valid) □

7.5 Correctness of Reconstruction

Theorem 2. *Our reconstruction function returns a valid troop allocation with the same amount of troops as the calculated minimum obtained from our dynamic-programming algorithm.*

Proof. Obviously, we will select some root troop amount $x \in \{0, 1, 2\}$ that minimises: min(dp[1][3][0], dp[1][3][1], dp[1][3][2]).

Then, during the execution of our dynamic-progamming algorithm, we will have saved for each state $dp[u][p][x]$, the best decisions made. Then, using this information, our reconstruction function will allocate x new soldiers at a new soldier at vertex u, and then utilise our decisions saved in order to know how many setStatusount of soldiers each of it's children will have. By looking at our dynamicprogramming algorithm's correctness, we can be convinced that these decisions stored were optimal due to them having minimum cost at each state. As well each vertex (except root) has only one parent. Momentarily we can now see how our decisions make sure that we visit each vertex once and that it has an appropriate amount of soldiers matching its parent's state. In this way,

We know that our reconstruc posture meets the conditions required and has a cost equal to our root decision chosen (which had to be optimal). Overall, this means our reconstruction function will return a valid allocation using minimum amount of soldiers. □

8 Complexity Analysis

Denote by n the number of nodes in the given tree:

8.1 Reading Input And Building List Of Adjacency Lists

The input contains exactly $n - 1$ edges.

For each edge (u, v), the programme adds two items to the adjacency lists:

$$v \in \texttt{adj[u]}, \qquad u \in \texttt{adj[v]}.$$

These additions takes amoritized $O(1)$ time. Thus in total it takes:

$$T_{\text{input}}(n) = \sum_{i=1}^{n-1} O(1) = O(n).$$

This creates an adjacency list of size $n + 1$ vectors holding $2(n - 1)$ entries and so the storage required is a linear function in n.

$$S_{\text{adj}}(n) = O(n).$$

8.2 DFS traversal

Each vertex is added to the stack only once. Each vertex is then processed once. When vertex u is processed, each element in its adjacency list is also push to be processed.

Let $\deg(u)$ be the degree of vertex u. The total number of elements processed in the adjacency lists will be:

$$\sum_{u \in V} \deg(u).$$

Using the handshaking lemma:

$$\sum_{u \in V} \deg(u) = 2|E|.$$

As the input graph is a tree:

$$|E| = n - 1.$$

$$\sum_{u \in V} \deg(u) = 2(n - 1) = O(n).$$

So the total time taken to process all vertices and their adjacency lists will be:

$$\boxed{T_{\text{DFS}}(n) = O(n)}.$$

The parent array, depth array, traversal order vector and DFS stack all take up a maximum

of $O(n)$ space.

$$\boxed{S_{\text{DFS}}(n) = O(n)}.$$

8.3 Reverse order

We store all n vertices in the vector `order`. Reversing a vector of size n can be done in linear time:

$$T_{\text{reverse}}(n) = O(n).$$

In-place reversal requires $O(1)$ additional space.

8.4 DP Table Initialization

Each vertex in the DP table has 4 possible parent states and 3 possible troop counts. The DP table will therefore have

$$4 \cdot 3 \cdot (n+1) = 12(n+1)$$

entries, where we also count the unused vertex index zero. Initializing all entries takes

$$\boxed{T_{\text{init}}(n) = O(n)}$$

time. We store a constant amount of data per vertex in the table, so

$$\boxed{S_{\text{DP}}(n) = O(n)}.$$

8.5 Complexity of the DP transitions

There are twelve states for each vertex. However, we need to be careful. We have to see how much work is being done for each state.

Denote by d_u the number of children of vertex u. Since each non-root vertex has one parent,

$$\boxed{\sum_{u \in V} d_u = n - 1}$$

This will be key to analyse the complexities.

8.5.1 Case 1: Vertex has 1 or 2 troops

When $x \in 1, 2$, the algorithm iterates through each child. We take the minimum of the 3 dp values

$$\min\{dp[v][x][0], dp[v][x][1], dp[v][x][2]\}.$$

This can be calculated in constant time. If u has d_u children, the work for a state is $O(d_u + 1)$. As the number of states per vertex is constant, the total time complexity would be:

$$\begin{aligned} T_1(n) &= O\left(\sum_{u \in V}(d_u + 1)\right) \\ &= O((n-1) + n) \\ &= O(n). \end{aligned}$$

By using $\sum_u d_u = n - 1$, this simplifies to

$$\boxed{T_1(n) = O(n)}.$$

8.5.2 Case 2: No troops. Parent has 2 troops

When $x = 0$ and $p = 2$, the algorithm iterates through each child once. This calculates the following

$$\min\{dp[v][0][0], dp[v][0][1]\}.$$

For each child, the algorithm performs constant work. The total time to do this for all vertices will be:

$$T_2(n) = O\left(\sum_{u \in V}(d_u + 1)\right) = O(n).$$

So,

$$\boxed{T_2(n) = O(n)}.$$

8.5.3 Case 3: 0 troops and parent does not have 2 troops

This is the case which causes the worst-case quadratic runtime.

In this case where $x = 0$ and $p \neq 2$ all children are treated as the single child with two troops. For each selected child, all other children are scanned.

Say vertex u has d_u children.

The outer loop will run d_u times. The inner loop will then check the remaining $d_u - 1$ children. This means that we will have

$$d_u(d_u - 1)$$

iterations of the inner loop. Adding in the outer loop we have

$$T_u = O(d_u + d_u(d_u - 1)) = O(d_u^2)$$

iterations at vertex u Adding this up across all vertices we get

$$\boxed{T_3(n) = O\left(\sum_{u \in V} d_u^2\right)}$$

Although the sum of children is linear, the sum of squares may not be. As all d_u are positive we can write

$$\left(\sum_{u \in V} d_u\right)^2 = \sum_{u \in V} d_u^2 + 2 \sum_{\substack{u,v \in V \\ u<v}} d_u d_v$$

All terms in the second sum are positive so

$$\sum_{u \in V} d_u^2 \leq \left(\sum_{u \in V} d_u\right)^2.$$

Using $\sum_u d_u = n - 1$, we obtain

$$\sum_{u \in V} d_u^2 \leq (n-1)^2.$$

Hence,

$$\boxed{T_3(n) = O(n^2)}.$$

We now prove that this is tight.

Imagine a star-shaped tree with a central root node, and having $n - 1$ children. All other nodes have 0 children.

When the root node is in a state with 0 troops and sentinel parent $p = 3$, the algorithm will reach this transition. In the outer loop, each of the $n - 1$ children will be considered. As a result, the inner loop will iterate through the remaining $n - 2$ children.

The inner loop will run

$$(n-1)(n-2) = n^2 - 3n + 2.$$

This means that $\Omega(n^2)$ work is needed on this one state in the star-shaped input. This leads to an upper bound and lower bound on the running time of this transition:

$$\boxed{T_3(n) = \Theta(n^2)}.$$

8.6 Reconstruction

The reconstruction function visits each vertex exactly once. At each vertex, it follows the troop-placement decisions stored during the dynamic programming computation. Since each vertex has exactly one parent, the total number of stored child choices traversed is $n-1$. Therefore, the total time complexity is

$$T_{\text{reconstruct}}(n) = O(n).$$

The troop allocation array requires $O(n)$ space. The recursive function uses $O(h)$ stack space, where h is the height of the tree. Since $h \leq n-1$, the auxiliary space complexity is

$$S_{\text{reconstruct}}(n) = O(n).$$

8.7 Overall time complexity

The main components have the following time bounds:

Component	**Time complexity**
Adjacency-list construction	$O(n)$
DFS traversal	$O(n)$
Reversing the traversal order	$O(n)$
DP table initialization	$O(n)$
Transitions for $x \in \{1, 2\}$	$O(n)$
Transitions for $x = 0,\ p = 2$	$O(n)$
Transitions for $x = 0,\ p \neq 2$	$O(n^2)$
Reconstruction	$O(n)$

$$T(n) = O(n) + O(n^2)$$

As n^2 grows faster than n,

$$T(n) = O(n^2)$$

We have a quadratic lower bound due to the star-shaped example. Therefore, the tight time complexity in the worst-case for this implementation is:

$$\boxed{T(n) = \Theta(n^2)}$$

8.8 Overall space complexity

The adjacency list takes up $O(n)$ space.

The parent array, depth array, traversal ordering and DFS stack all take up at most $O(n)$ space.

The DP table contains 12 entries per vertex (plus a constant factor), thus takes up $O(n)$ space.

No further data structures are created in the algorithm that are superlinear in

terms of n. Therefore,

$$S(n) = O(n) + O(n) + O(n) + O(n)$$

Overall this means:

$$\boxed{S(n) = O(n)}$$

So this algorithm uses linear memory despite having a quadratic running time in the worst case.

9 Conclusion

We present a dynamic programming solution to determine the minimum number of soldiers needed. The unique path property ensures the graph is a tree. Using a root we can split our problem into smaller subproblems. Dynamic programming states keep track of the number of soldiers at a given vertex and the parent vertex. The transitions depend on whether the current vertex has 1 or 2 soldiers. If the vertex has 0 soldiers then we need to be careful when considering the number of neighboring vertices with 2 soldiers. We prove the correctness using induction on the height of each subtree. This ensures we find the optimal value for each DP state. We then choose the minimum value between the 3 possible soldier values at the root. The code implemented uses adjacency lists, iterative DFS and DP state array. In the worst case the algorithm has a time complexity of $\Theta(n^2)$ due to having to iterate through the child with 2 soldiers multiple times and having to re-iterate through the remaining children. The space complexity is $O(n)$. The project demonstrates how the graph structure, tree traversal, dynamic programming, proving correctness and asymptotic analysis can be applied to find a constrained combinatorial optimisation solution.